

DK WAVE TECHNOLOGY

Laboratoire Azure — Administrateur Systèmes & Réseaux Cloud

RAPPORT DE PHASE 3

Sécurité & Gouvernance

Date : 25 Avril 2026 | Rédigé par : Lancelot DOSSOU | Environnement : Azure Subscription 1

RÉSUMÉ EXÉCUTIF

La Phase 3 du laboratoire Azure DK WAVE TECHNOLOGY a consisté à sécuriser en profondeur l'infrastructure déployée en Phase 2. Partant d'un environnement fonctionnel mais sans protection active, nous avons mis en place une architecture de sécurité multicouche couvrant la surveillance continue, la protection des secrets, le contrôle des accès, l'inspection du trafic réseau, la protection applicative et la gouvernance automatisée. À l'issue de cette phase, l'infrastructure DK WAVE TECHNOLOGY est conforme aux standards entreprise et alignée avec le modèle Zero Trust.

1. Contexte du projet

DK WAVE TECHNOLOGY est une entreprise éditrice de logiciels SaaS, comptant 20 employés en télétravail, sans centre de données physique. L'intégralité de l'infrastructure IT repose sur Microsoft Azure.

À la fin de la Phase 2, l'infrastructure disposait d'un réseau virtuel segmenté (VNet 10.0.0.0/16), de deux machines virtuelles privées (vm-web01 et vm-app01), d'un accès administrateur sécurisé via Azure Bastion, et des bases de la haute disponibilité. Cependant, cette infrastructure présentait plusieurs lacunes critiques du point de vue de la sécurité :

- Aucune surveillance active de la posture de sécurité — les failles n'étaient pas détectées
- Les secrets applicatifs (mots de passe, clés) pouvaient être stockés en clair dans des scripts
- Aucun mécanisme ne contrôlait le trafic réseau sortant — une VM compromise pouvait librement communiquer avec l'extérieur
- Aucune protection applicative contre les attaques web (injection SQL, XSS, etc.)
- Aucune politique automatisée ne bloquait les mauvaises configurations
- Aucune protection contre les suppressions accidentelles de ressources critiques

Principe directeur de la Phase 3 : Zero Trust — ne jamais faire confiance, toujours vérifier. Chaque accès, chaque flux réseau, chaque configuration est contrôlé et audité, quelle que soit l'origine de la demande.

2. Objectifs de la Phase 3

Les objectifs définis au démarrage de cette phase étaient les suivants :

- Activer une surveillance continue et automatique de la posture de sécurité via Microsoft Defender for Cloud
- Centraliser tous les secrets applicatifs dans un coffre-fort sécurisé et audité
- Appliquer le principe du moindre privilège sur l'accès aux secrets
- Inspecter et filtrer tout le trafic réseau sortant via Azure Firewall
- Forcer l'intégralité du trafic à transiter par le Firewall via des routes personnalisées
- Protéger la couche applicative contre les attaques web via un WAF en mode Prevention
- Bloquer automatiquement les mauvaises configurations via Azure Policy
- Prévenir les suppressions accidentelles de ressources critiques via des Resource Locks

3. Réalisations détaillées

3.1 — Microsoft Defender for Cloud

Contexte

Avant cette étape, l'infrastructure Azure n'était surveillée par aucun outil de sécurité centralisé. Les éventuelles failles ou comportements suspects passaient totalement inaperçus.

Ce qui a été fait

- Activation du plan Defender pour les Serveurs (Plan 2) — couvre vm-web01 et vm-app01
- Activation du plan Defender pour Key Vault — surveille les accès au coffre-fort
- Vérification de la couverture : statut Complet sur les 2 serveurs détectés
- Le Secure Score est maintenant visible et calculé automatiquement

Pourquoi c'est important

Defender for Cloud est le système d'alarme permanent de l'infrastructure. Il analyse en continu la configuration de toutes les ressources Azure, détecte les comportements suspects, génère des alertes de sécurité et produit un Secure Score — une note chiffrée qui mesure la posture de sécurité globale. Sans cet outil, l'équipe IT est aveugle face aux menaces.

Impact business : Defender for Cloud permet à DK WAVE TECHNOLOGY de détecter les incidents de sécurité avant qu'ils n'impactent les utilisateurs, réduisant le MTTR (Mean Time To Respond) et limitant l'exposition aux risques.

3.2 — Azure Key Vault (kv-dkwave-secure)

Contexte

Avant cette étape, les mots de passe et secrets applicatifs (connexion base de données, clés API) étaient potentiellement stockés en clair dans des scripts ou des variables d'environnement — pratique extrêmement risquée en production.

Ce qui a été fait

- Création du Key Vault kv-dkwave-secure dans rg-dkwave-prod, région West Europe
- Choix du modèle d'autorisation RBAC Azure — plus sécurisé que les stratégies d'accès classiques
- Création d'un Private Endpoint (pe-kv-dkwave) dans subnet-app — le coffre est invisible depuis Internet
- Désactivation totale de l'accès public — aucune connexion Internet directe possible
- Intégration DNS privée via la zone privatelink.vaultcore.azure.net — résolution automatique depuis le VNet
- Ajout du secret DbPassword simulant le mot de passe de base de données de production
- Application des tags de gouvernance sur le Key Vault ET le Private Endpoint

Pourquoi c'est important

Le Key Vault avec Private Endpoint crée une situation où le coffre-fort n'existe pas sur Internet — il n'est joignable que depuis l'intérieur du réseau privé Azure. Un attaquant externe, même connaissant l'URL du coffre, ne peut pas l'atteindre. Seules les VMs dans subnet-app peuvent y accéder.

Difficulté rencontrée : Le nom kv-dkwave-prod était déjà réservé par un coffre supprimé en mode Soft Delete (rétention 90 jours). Solution : utilisation du nom kv-dkwave-secure. Leçon : toujours vérifier les noms disponibles avant de planifier un déploiement.

3.3 — Least Privilege sur le Key Vault

Contexte

Avoir un coffre-fort sécurisé ne sert à rien si tout le monde en a la clé. Il faut définir précisément qui peut faire quoi, et accorder uniquement les droits strictement nécessaires à chaque identité.

Ce qui a été fait

Identité	Rôle attribué	Périmètre	Permissions
Lancelot DOSSOU	Key Vault Administrator	kv-dkwave-secure	Gestion complète
GRP-AZ-Ops (Bob)	Key Vault Secrets User	kv-dkwave-secure	Lecture seule des secrets
Autres identités	Aucun rôle	kv-dkwave-secure	Accès refusé

- Attribution du rôle Key Vault Administrator à l'administrateur principal — gestion complète du coffre
- Attribution du rôle Key Vault Secrets User au groupe GRP-AZ-Ops via CLI — lecture seule
- Validation : tentative d'accès sans rôle → refusée par RBAC

Pourquoi c'est important

Bob (GRP-AZ-Ops) a besoin de lire le mot de passe de base de données pour faire son travail, mais il n'a aucun besoin de le modifier ou d'en créer de nouveaux. En lui donnant Key Vault Secrets User et non Administrator, on réduit le blast radius en cas de compromission de son compte : un attaquant qui prendrait le contrôle du compte de Bob ne pourrait que lire les secrets, pas les effacer.

Difficulté rencontrée : Après création du Key Vault en mode RBAC, même l'administrateur se voyait refuser l'accès aux secrets (erreur RBAC). Le modèle RBAC Azure ne donne aucun accès implicite au créateur — il faut explicitement s'attribuer un rôle. Solution : attribution manuelle du rôle Key Vault Administrator via IAM.

3.4 — Azure Firewall (azfw-dkwave)

Contexte

Les NSG de la Phase 2 filtraient le trafic entrant par port et protocole. Mais aucun mécanisme ne contrôlait le trafic sortant — une VM compromise pouvait librement communiquer avec n'importe quel serveur externe, exfiltrer des données, ou recevoir des instructions d'un serveur de commande et contrôle.

Ce qui a été fait

- Création du sous-réseau AzureFirewallSubnet (10.0.0.0/24) — nom imposé par Azure
- Création de l'IP publique pip-azfw (20.67.106.59) en SKU Standard
- Déploiement du Firewall azfw-dkwave avec IP privée 10.0.0.4
- Association du Firewall au VNet vnet-dkwave-prod via la configuration fw-ipconfig

Pourquoi c'est important

L'Azure Firewall opère au niveau réseau et applicatif. Il inspecte tout le trafic qui le traverse, journalise chaque connexion (source, destination, port, protocole), et peut bloquer les connexions vers des domaines malveillants connus via Threat Intelligence. C'est la différence entre un portier qui vérifie les identités et une simple barrière qui s'ouvre automatiquement.

Difficulté rencontrée : La commande `az network firewall create` a nécessité l'installation de l'extension `azure-firewall` non présente par défaut. De plus, le paramètre `--sku-tier` n'était pas reconnu dans la version installée. Solution : installation de l'extension + suppression du paramètre non supporté.

3.5 — User Defined Routes (UDR)

Contexte

Déployer un Firewall ne suffit pas. Par défaut dans Azure, le trafic des sous-réseaux ne passe pas par le Firewall — il suit les routes système d'Azure. Il faut explicitement forcer le trafic à transiter par le Firewall via des tables de routage personnalisées.

Ce qui a été fait

- Création de la table de routage `rt-dkwave-fw` dans `rg-dkwave-shared`
- Création de la route `DefaultRoute` : `0.0.0.0/0` → `VirtualAppliance` → `10.0.0.4`
- Association de la table à `subnet-web` (`10.0.10.0/24`)
- Association de la table à `subnet-app` (`10.0.20.0/24`)

Pourquoi c'est important

La route `0.0.0.0/0` signifie tout le trafic sans exception. En indiquant `VirtualAppliance` avec l'IP privée `10.0.0.4` (notre Firewall), on oblige Azure à faire passer toutes les requêtes sortantes par le Firewall, qui les inspecte avant de les laisser passer ou de les bloquer. C'est l'UDR qui donne au Firewall son rôle réel.

Analogie : Le Firewall est le douanier. Les UDR sont les panneaux de signalisation qui obligent tous les véhicules à passer par le poste de douane. Sans les panneaux, les véhicules peuvent contourner le douanier par des routes alternatives.

3.6 — Application Gateway avec WAF (appgw-dkwave)

Contexte

Le Firewall Azure protège le trafic réseau (couche 3-4). Mais les attaques web les plus courantes — injections SQL, Cross-Site Scripting, traversée de répertoire — se produisent dans le contenu des requêtes HTTP, que le Firewall ne peut pas analyser. Il fallait une protection au niveau applicatif (couche 7).

Ce qui a été fait

- Création du sous-réseau subnet-appgw (10.0.50.0/24) dédié à l'Application Gateway
- Création de l'IP publique pip-appgw (20.101.48.244) en SKU Standard
- Création de la WAF Policy waf-dkwave avec le ruleset Microsoft_DefaultRuleSet 2.1
- Déploiement de l'Application Gateway appgw-dkwave en SKU WAF_v2
- Activation du WAF en mode Prevention — les attaques détectées sont bloquées, pas seulement journalisées
- Activation du requestBodyCheck — inspection du corps des requêtes HTTP

La différence Detection vs Prevention

En mode Detection, le WAF observe les attaques et les journalise mais laisse passer le trafic malveillant. En mode Prevention, le WAF bloque activement toute requête correspondant à un pattern d'attaque connu. En production, le mode Prevention est le seul acceptable.

Difficulté rencontrée : L'Application Gateway cherchait le subnet dans rg-dkwave-prod alors que le VNet est dans rg-dkwave-shared. Azure CLI ne résolvait pas la référence cross-resource-group avec le nom court. Solution : utilisation de l'ARM Resource ID complet du subnet incluant le chemin /subscriptions/.../resourceGroups/rg-dkwave-shared/...

3.7 — Azure Policy

Contexte

Même avec des règles RBAC et des NSG en place, rien n'empêchait un administrateur de déployer accidentellement une VM avec une IP publique, ou de créer une ressource sans les tags obligatoires. Il fallait un mécanisme automatique qui bloque ces erreurs à la source.

Ce qui a été fait

Nom de la policy	Rôle	Périmètre	Effet
deny-public-ip	Interdire les IPs publiques sur les VMs	Toute la subscription	Deny
require-tags	Exiger le tag Environment sur toutes les ressources	Toute la subscription	Deny

- Les deux policies sont assignées sur toute la subscription — elles s'appliquent à toutes les ressources créées désormais
- La policy deny-public-ip bloque la création de toute VM avec une IP publique assignée à sa NIC
- La policy require-tags refuse la création de toute ressource sans le tag Environment

Pourquoi c'est important

Azure Policy est la gouvernance automatisée. Contrairement au RBAC qui contrôle qui peut faire quelque chose, Azure Policy contrôle ce qui peut être fait. Même un Owner avec tous les droits ne peut pas créer une ressource qui viole une policy Deny. C'est le dernier filet de sécurité contre les erreurs de configuration.

3.8 — Resource Locks

Contexte

Un administrateur pressé, une mauvaise manipulation, une commande CLI mal formée — n'importe quelle de ces situations peut entraîner la suppression accidentelle d'un groupe de ressources entier, emportant avec lui toutes les VMs, le réseau, les configurations. Il fallait un mécanisme de protection absolu.

Ce qui a été fait

- Création du lock lock-shared-rg (type CanNotDelete) sur rg-dkwave-shared
- Création du lock lock-prod-rg (type CanNotDelete) sur rg-dkwave-prod

Pourquoi c'est important

Un lock de type CanNotDelete empêche physiquement la suppression d'une ressource ou d'un groupe de ressources, quelle que soit la permission RBAC de l'utilisateur — même un Owner ne peut pas supprimer une ressource verrouillée. Pour le faire, il faut d'abord retirer le lock, action qui laisse une trace dans les logs d'audit et crée une friction intentionnelle. En entreprise, ce délai forcé permet d'identifier et d'annuler une suppression accidentelle avant qu'elle ne soit effective.

4. Difficultés rencontrées et solutions appliquées

La Phase 3 a été la plus complexe techniquement du lab, avec six difficultés majeures rencontrées et résolues.

Difficulté 1 — Nom de Key Vault réservé par le Soft Delete

Lors de la tentative de création du Key Vault avec le nom kv-dkwave-prod, Azure a retourné une erreur indiquant que le nom était déjà utilisé ou réservé par un coffre précédemment supprimé.

Explication technique : Azure Key Vault dispose d'une fonctionnalité appelée Soft Delete, activée par défaut, qui conserve les coffres supprimés pendant 90 jours avant de les purger définitivement. Durant cette période, le nom reste globalement réservé et ne peut pas être réutilisé.

Solution appliquée : Utilisation du nom alternatif kv-dkwave-secure. Cette situation a été l'occasion d'expliquer que le Soft Delete est en réalité une protection précieuse — elle permet de récupérer un coffre supprimé accidentellement dans les 90 jours.

Difficulté 2 — Accès RBAC refusé sur le Key Vault au créateur

Immédiatement après la création du Key Vault avec le modèle d'autorisation RBAC Azure, la navigation vers la section Secrets retournait l'erreur : L'opération n'est pas autorisée par RBAC.

Explication technique : Avec le modèle RBAC Azure, il n'existe aucun accès implicite basé sur la propriété de la ressource. Même le compte qui a créé le Key Vault n'a aucun droit dessus tant qu'un rôle ne lui est pas explicitement attribué. C'est une différence fondamentale avec l'ancien modèle Vault Access Policy.

Solution appliquée : Attribution manuelle du rôle Key Vault Administrator au compte administrateur via la section Contrôle d'accès (IAM) du Key Vault. Cette situation illustre parfaitement le principe Zero Trust en action.

Difficulté 3 — Accès réseau bloqué après désactivation de l'accès public

Après avoir désactivé l'accès public du Key Vault et configuré le Private Endpoint, le portail Azure lui-même ne pouvait plus afficher les secrets. Le message indiquait : L'accès réseau public est désactivé et la demande ne provient pas d'un service approuvé.

Explication technique : La sécurité mise en place était si efficace qu'elle bloquait même le navigateur de l'administrateur. Le portail Azure est un service externe qui communique avec les APIs Azure depuis Internet — et cette connexion était désormais bloquée.

Solution appliquée : Configuration en trois points — autorisation des réseaux sélectionnés (au lieu de désactivation totale), ajout du VNet vnet-dkwave-prod dans la liste des réseaux autorisés, activation de l'exception pour les services Microsoft de confiance, et ajout de l'adresse IP publique du poste administrateur (176.187.44.206) dans les règles de pare-feu du Key Vault.

Difficulté 4 — Sous-réseau AzureFirewallSubnet absent

Lors de la tentative d'association du Firewall au VNet, Azure a retourné l'erreur InvalidResourceReference indiquant que le sous-réseau AzureFirewallSubnet n'existait pas dans le VNet.

Explication technique : Azure Firewall exige un sous-réseau nommé exactement AzureFirewallSubnet (casse comprise) avec un minimum de /26 (64 adresses). Ce sous-réseau n'avait pas été créé en Phase 2 — une lacune dans la planification initiale.

Solution appliquée : Création du sous-réseau AzureFirewallSubnet avec le préfixe 10.0.0.0/24, puis relance de la commande d'association. Leçon retenue : tous les sous-réseaux requis par des services managés Azure doivent être créés dès la Phase 2, avant même que les services ne soient déployés.

Difficulté 5 — Extension Azure Firewall CLI non disponible

La commande `az network firewall create` a déclenché une demande d'installation de l'extension `azure-firewall`, non présente par défaut dans Azure Cloud Shell. De plus, le paramètre `--sku-tier Standard` générait une erreur `unrecognized arguments`.

Solution appliquée : Installation de l'extension à la demande via la confirmation interactive, puis suppression du paramètre `--sku-tier` qui n'était pas supporté dans cette version de l'extension (Azure applique le tier `Standard` par défaut).

Difficulté 6 — Référence cross-resource-group dans l'Application Gateway

La création de l'Application Gateway échouait avec `InvalidResourceReference` car le subnet `subnet-appgw` était dans `rg-dkwave-shared` tandis que l'Application Gateway était créée dans `rg-dkwave-prod`. Azure CLI tentait de résoudre le nom du subnet dans le même resource group que l'Application Gateway.

Solution appliquée : Utilisation de l'ARM Resource ID complet du subnet au lieu du nom court, forçant Azure à chercher le subnet dans le resource group correct, indépendamment de là où l'Application Gateway est déployée.

5. Architecture de sécurité déployée

À l'issue de la Phase 3, l'architecture de sécurité multicouche suivante est en place :

Internet (Attaquant externe)



6. Tableau récapitulatif

Étape	Réalisation	Statut
3.1 — Defender for Cloud	Plans Servers et Key Vault activés — 2 serveurs surveillés	✅ Complété
3.2 — Secure Score	Score visible, recommandations listées et analysées	✅ Complété
3.3 — Azure Key Vault	kv-dkwave-secure + Private Endpoint + DNS privé + Secret DbPassword	✅ Complété
3.4 — Least Privilege	GRP-AZ-Ops → Secrets User Administrateur → KV Administrator	✅ Complété
3.5 — Azure Firewall	azfw-dkwave déployé — IP privée 10.0.0.4, IP publique 20.67.106.59	✅ Complété
3.6 — UDR	rt-dkwave-fw : 0.0.0.0/0 → 10.0.0.4 sur subnet-web et subnet-app	✅ Complété
3.7 — WAF	appgw-dkwave WAF_v2 — mode Prevention, ruleset DefaultRuleSet 2.1	✅ Complété

3.8 — Azure Policy	deny-public-ip + require-tags sur toute la subscription	☑ Complété
3.9 — Resource Locks	CanNotDelete sur rg-dkwave-shared et rg-dkwave-prod	☑ Complété

7. Conclusion et transition vers la Phase 4

La Phase 3 a été la phase la plus dense et la plus exigeante du laboratoire Azure DK WAVE TECHNOLOGY. En neuf étapes, nous avons transformé une infrastructure fonctionnelle mais exposée en un environnement de production sécurisé, gouverné et conforme aux standards entreprise.

Les six difficultés rencontrées — toutes résolues — ont permis d'approfondir la compréhension des mécanismes de sécurité Azure : le comportement du modèle RBAC, les contraintes réseau du Private Endpoint, les noms réservés par le Soft Delete, les exigences de nommage des sous-réseaux Azure, et la gestion des ressources cross-resource-group.

L'infrastructure est désormais protégée par cinq couches de sécurité distinctes et complémentaires : WAF applicatif, Azure Firewall réseau, NSG par sous-réseau, isolation des VMs sans IP publique, et accès aux secrets via Private Endpoint exclusivement. Cette architecture en défense en profondeur garantit qu'un attaquant doit franchir cinq barrières indépendantes pour atteindre les données.

La transition vers la Phase 4 — Monitoring & Operations — est naturelle : l'infrastructure sécurisée en Phase 3 génère maintenant des logs précieux (Firewall, WAF, Key Vault, NSG, Defender) qu'il faut centraliser, analyser et transformer en alertes proactives.

Bilan Phase 3 : 9 étapes réalisées, 6 difficultés surmontées, 5 couches de sécurité déployées, 0 compromis sur la posture de sécurité. DK WAVE TECHNOLOGY dispose désormais d'une infrastructure conforme au modèle Zero Trust, prête pour un audit entreprise.

DK WAVE TECHNOLOGY – Phase 3 complétée le 25 Avril 2026
Auteur : Lancelot DOSSOU – Administrateur Systèmes & Réseaux Cloud